AI in Science: Workflows

D-Lab AI Pulse, Fall 2026: Session 5

Materials: `https://github.com/dlab-berkeley/AI_Pulse`
Workshop suggestions: `https://forms.gle/x8KfejgCViE7t6iQA`

27 October 2026

This Session

- Today is a wide view: six cases from the past year, no tool to follow along with
- Each case is a researcher who handed part of a real project to an AI
- Six fields: genomics, political science, ecology, materials, cosmology, mathematics
- The focus is the method, not the application: what they did, not what they found
- The idea is to understand how an LLM can perform a particular task systematically

How Each Case Runs

- Context: who, where, when
- The problem: what question is asked, and why it was hard before these models
- The setup: exactly how the model is prompted, fed, checked and looped
- The results: what they got, what it cost, and the limitations
- The method: the same thing with the field removed, as a recipe for your own work
- One named technique per case, so you can look it up and recognize it elsewhere

The Six Cases

Field	The example	The method
Genomics	Labelling 9,000 experiments	Extraction with evidence, then an audit
Political science	Coding political texts	Model disagreement as a signal of ambiguity
Ecology	Screening 19,000 papers	Ranking, with a stopping rule set in advance
Materials	Growing a crystal film	Prediction before each run, then refutation
Cosmology	Searching dark energy models	Constrained search with deterministic checks
Mathematics	Six open problems in five days	A full specification, with adversarial checks

Last Spring: Six Examples, Six Methods

Field	The example	The method
Astronomy	Real transients vs noise	15 explained examples, coherence as a flag
Biology	Cancer drug pairs	Hypothesize, test in the lab, feed back
Social science	Lives from childhood essays	Embed the text, predict with ordinary ML
Physics	A 40-year conjecture	Simplify to small cases, ask for the pattern
Chemistry	Reaction conditions	Fine-tune a small model on 1.28M examples
Mathematics	Theorems in Lean	Blueprint, decompose, iterate

Slides: `https://github.com/dlab-berkeley/AI_Pulse/tree/main/Previous-Workshops/Spring-2026/2026-03-31_AI_in_Science_Case_Studies`

Case 1

Genomics: labelling 9,000 experiments

Context

- Gemma, at the University of British Columbia, is a curated gene expression database
- Each experiment needs standard labels: mouse strain, cell line, from a fixed list
- Curators have done this by hand for years, for more than 9,000 experiments
- Pavlidis and colleagues, March 2026, now peer reviewed
- A companion from a Delhi liver clinic, June 2026, does the same on discharge notes

Paper: https://www.biorxiv.org/content/10.64898/2026.03.05.709906v2 Delhi:

https://www.medrxiv.org/content/10.64898/2026.06.06.26354838v1

The Problem

- The input is messy: free-text metadata and a methods section mentioning the strain
- Tested on 9,000 experiments that already had human labels, so it could be checked
- Pattern matching gets 6% of experiments fully right. Curators get most, slowly
- And the curators' own labels have errors nobody has had time to look for
- The question: can a model label at scale, and can you tell when it is wrong, fast?

The Setup

- One model, fixed randomness, a fixed output schema, the methods section as input
- Every label must come with a verbatim quote from the text
- Strains: 156 terms fit in a prompt. Cell lines: 46,032, so search, then pick from 50
- Output is compared automatically with the labels curators had already assigned
- Every disagreement goes to a curator, who decides which side was wrong
- Delhi adds a layer: reason, then fill the schema, then quote a span for each value

Results and Limitations

- Strains: 77% of experiments fully correct, against 6% for pattern matching
- Reviewing the disagreements found 230 errors in labels humans had trusted for years
- The quote is what makes review fast: the evidence arrives with the label
- It still invented a strain not in the text. The quote was honest, so it was caught
- Cell lines were harder: errors at naming, at retrieval, and at the final pick
- Delhi: 99% on 2,250 values, but on simulated notes, with the reference edited after
- One model, one run, and a quote proves the source exists, not the label

The Method: Extraction With Evidence

1. Decide the fields you want out, and allow an answer of "not stated"
2. Make the model return the evidence with every answer: the passage it used
3. Compare its answers with a set you already trust, or make a small one by hand
4. Look only at the disagreements, and decide who was right, in both directions
5. Sort the errors by cause and by cost
6. Correct the trusted set where the model turned out to be right

Works for any extraction from documents: charts, surveys, field notes, archives

Named Technique: Structured Output

- Instead of prose, the model fills a form you define: fields, types, allowed values
- Every API offers it, as JSON schema, response format or structured output
- In a chat, the cheap version is asking for a table with named columns
- It makes answers comparable across thousands of documents, and checkable by code
- Add a field for "not stated", or the model will fill the blank with a guess

Case 2

Political science: coding political texts

Context

- Political scientists code texts by hand: is this speech populist, this post hostile?
- A codebook defines the categories. Coders apply it. Their agreement is the check
- Three Harvard PhD students asked whether a model can replace the coders, and when
- August 2026, presented at the field's methods conference
- A companion from Amsterdam used the same signal to grade about 6,000 student answers

Paper: `https://arxiv.org/abs/2609.22133` Amsterdam: `https://www.mdpi.com/2504-4990/8/3/74`

The Problem

- Experts agree only 67% of the time on the hard tasks. So what is "correct"?
- One model in place of the coders gives one answer per text, and no sense of doubt
- Crowdworkers are cheap but noisy. Experts are reliable but cost weeks
- The hard texts are hard because the codebook is ambiguous, not the coders weak
- The question: can you find the ambiguous texts and rules without recoding it all?

The Setup

- Fourteen published studies, 100 texts each, re-coded with the original codebooks
- Ten models, from one small enough for a laptop to the largest commercial ones
- Three experts and 165 crowdworkers coded the same texts, for comparison
- For every text, how much the ten models disagree is recorded as a number
- One codebook was rewritten with explicit rules, and humans and models coded again
- Amsterdam's variant: three models each grade an answer 20 times; take the mode

Results and Limitations

- On easy tasks, all but the smallest model match the agreement between experts
- On hard tasks, nine of ten models agree with an expert more often than experts do
- Model disagreement tracks expert disagreement: it is a usable flag for ambiguity
- Clarifying one codebook raised expert agreement from 0.64 to 0.82. Blame the rules
- Amsterdam: only low-certainty answers went to a human, and marking fell 40 to 90%
- Small models got worse with the clearer codebook; the loop was tried once
- Agreement is not accuracy, and models fed the same web share blind spots

The Method: Disagreement as a Signal

1. Write the codebook, and hand-code a small sample as you would anyway
2. Run several cheap models on a pilot, or one model many times at non-zero randomness
3. Where they agree, move on. Where they disagree, read the text yourself
4. The disagreements point at codebook rules that are not doing their job. Fix them
5. Repeat until disagreement stops responding to clearer rules. What is left is real
6. Code the full corpus, and report a range: ambiguous items counted both ways

Works for any coding or grading with a rubric: interviews, open answers, student work

Named Technique: Self-Consistency

- The same question, asked many times with randomness on, gives a spread of answers
- The most common answer is usually better than any single run
- How often it wins is a confidence score you did not have before
- Low agreement marks the items to look at yourself, and the instructions to fix
- It costs N calls instead of one, so use it on the pilot, or on the hard items only

Case 3

Ecology: screening 19,000 papers

Context

- A systematic review starts with a search returning thousands of papers, most useless
- Someone reads every title and abstract. For a big review, weeks of a postdoc's time
- Missing a relevant paper is the failure. Reading irrelevant ones is only a cost
- A CSIRO and University of Tasmania team, July 2026, released as an R package
- Tested on ten finished ecology reviews where every record had been judged by hand

Paper: https://www.biorxiv.org/content/10.64898/2026.07.29.741127v2

The Problem

- Relevant papers are 2% to 25% of the pile. A tool that drops a few is useless
- Ask a model "include or exclude?" and you get verdicts with no idea what it missed
- Tried as a classifier, it agreed with humans at kappa 0.23. Humans manage about 0.8
- Active-learning tools work, with a person screening as the tool learns
- The question: how do you use an unreliable judge without ever trusting its verdict?

The Setup

- The review's own criteria go into the prompt. The model scores each one, 0 to 100
- Five small open models on a local machine each score every record three times
- The scores are averaged into one ranking. The model never says include or exclude
- A person reads a random 200 records first, to estimate how many hits exist
- Then they read down the ranking until a rule written in advance says stop
- The rule: half the pile read, fifty rejects in a row, and the estimate satisfied

Results and Limitations

- At least 95% of relevant papers recovered on all ten reviews, 98% on average
- While reading 56% of the pile, on average
- Workload on par with the usual active-learning tool: 39% vs 42% at 95% recall
- Of 50 contested labels re-screened by an expert, 14 changed, 8 were over-inclusions
- As a classifier the same ensemble is bad. The ranking is what works
- The rule needs the spot check. Without the 200 records, no recall figure
- More models did not help. Four small ones beat five

The Method: Ranking With a Stopping Rule

1. Turn your criteria into separate questions and have the model score each, not decide
2. Use several small models and repeat, so one model's quirks wash out
3. Average into one ranking, best candidates first
4. Before reading, take a random sample by hand to estimate how many hits exist
5. Write the stopping rule down now: coverage, a run of misses, the estimate met
6. Read from the top until the rule fires. Report recall against your sample

Works for any triage where a miss costs more than extra reading: records, images, alerts

Named Technique: Small Models, Run Locally

- Open-weight models of 10 to 30 billion parameters run on a good laptop or a desktop
- Tools like Ollama install one with a single command, and nothing leaves the machine
- They are weaker than the frontier models, but for scoring and sorting that is enough
- Five of them, run three times each over 19,000 records, cost nothing but electricity
- For sensitive data, or a task you will run a million times, try this first

Case 4

Materials: growing a crystal film

Context

- Thin films are grown by sputtering a target in vacuum. One knob is temperature
- A University of Tokyo group with an instrument maker, September 2026, code released
- The material: lithium cobalt oxide, the stuff of battery cathodes, as a crystal film
- The agent ran the sputtering chamber, the X-ray diffractometer and the microscope
- Eighteen experiments in one campaign, with no human intervention

Paper: `https://arxiv.org/abs/2609.18598` Code: `https://github.com/izumitkhr/synagent`

The Problem

- Find the temperature window where the film comes out crystalline and well oriented
- The usual way: sweep the knob, or let an optimizer pick the next point. Either works
- Neither tells you why, and the why is what you need for the next material
- Automated labs can run the loop, but chasing good results learns nothing from bad
- The question: can the loop be made to produce an explanation as well as an optimum?

The Setup

- The agent keeps a written document of what it currently believes about the material
- Before each run it writes a prediction: the temperature, and the outcome as a number
- It labels the run: this one should work, or this one should fail
- The chamber runs. The X-ray and microscope data come back
- If it lacks an analysis, a helper agent writes the code, tests it, and files it
- The document is rewritten every cycle. A miss forces a real revision, not a tweak

Results and Limitations

- Eighteen runs found the window: a threshold between 630 and 650 degrees, best at 670
- Three predictions were refuted, at runs 1, 2 and 10. Each forced a real revision
- The run meant to fail at 700 degrees succeeded, and that overturned what it believed
- The output is a document a human can read and argue with, not only a number
- "Refuted" was a yes or no call by the model, with no tolerance on the number
- One knob, one material, no repeats, no head-to-head with an ordinary optimizer
- The microscope could not tell what the bright particles were, nor could the agent

The Method: Prediction and Refutation

1. Have the model write down what it currently thinks, as a document you can read
2. Before each experiment or analysis, make it commit to a number, in writing
3. Alternate runs it expects to pass with runs it expects to fail. Both are information
4. Compare the result to the prediction. Agreement is cheap. A miss triggers a rewrite
5. When an analysis is missing, have it build the tool, test it, and keep it
6. The document at the end is the main result, alongside the optimum

Works for any sequence of experiments, simulations or model fits, in a lab or on a laptop

Named Technique: The Agent Loop

- An agent is a model in a loop: plan, act through a tool, observe, revise, repeat
- A tool is anything the model can call: a script, a database, an instrument
- The coding agents many of you use are this loop, with the file system as the tool
- Here the tool was a growth chamber, and the observation was a diffraction pattern
- The loop runs without you, so what you write down beforehand is the only control

Case 5

Cosmology: searching for dark energy models

Context

- The expansion of the universe is speeding up. The standard model uses a constant
- Theorists have proposed hundreds of alternatives, each a different equation
- Testing one against the supernova and galaxy surveys is routine. Thousands is not
- A Brown University group, posted 12 October, revised 23 October
- Eight different models each made a thousand picks from the candidates

Paper: `https://arxiv.org/abs/2610.16536`

The Problem

- The space of candidate theories is effectively infinite. Humans try a few at a time
- Code can enumerate thousands of legal equations. Choosing which to test is the work
- Most are unusable: unstable, inconsistent, or an old theory in a new form
- Nothing the model chooses can be trusted on its own. Each choice has to pass checks
- The model should choose what to test. Only the code should say whether it works

The Setup

- The humans wrote a grammar: 141 building blocks, and rules for combining them
- The code enumerates every legal equation. The model picks which one to test next
- Each proposal then passes a chain of checks written by physicists, not by the model
- Can the equations be derived? Are they stable? Do they integrate? Then: the data
- Scores reweight the candidate list, and the model picks again from the new order
- At the end, humans check whether the winners are new or old theories in disguise

Results and Limitations

- All eight models found theories that fit the data a little better than the standard
- Seven of the eight ended up in the same corner of the space
- About two thirds of what the models picked was rejected before it reached the data
- The authors are careful: the gains are small and do not unseat the standard model
- The authors say the gates do not check every physical requirement
- The grammar is redundant, so the same theory recurs in different forms
- Two weeks old. Nobody outside the group has checked it yet

The Method: Constrained Search

1. Define the space it may choose from, narrow enough that you can check anything in it
2. Have code enumerate the candidates, and let the model choose, with a reason, in bulk
3. Put deterministic gates between proposal and score: code that cannot be argued with
4. Score the survivors with the calculator you already trust
5. Feed the scores back and let it propose again
6. Record two numbers: how much gets rejected, and whether different models converge

Works wherever a trusted calculator exists: a simulator, a solver, a test suite

Named Technique: Constrained Generation

- You can restrict what a model may output: a list, a grammar, a fixed set of options
- The strict form: code enumerates the legal choices and the model only picks one
- Nothing outside the list can appear, so a plausible invention is no longer possible
- The model's judgment is still used, on the choice and on the reason for it
- The price is that you have to be able to write down the space in advance

Case 6

Mathematics: six open problems in five days

Context

- The Erdős problems: hundreds of open questions, numbered, each with a forum
- Shouqiao Wang, a PhD student at Columbia's business school, not its maths department
- Six solved out of about thirteen attempted: a 46% success rate, by his own count
- Solutions took six to thirty-two hours on a coding agent, unattended
- He published everything: prompts, proofs, code, and the machine-checked versions

Thread: `https://x.com/Qiaoqiao2001/status/2080003441821163958` Repository: `https://github.com/ShouqiaoW/erdos`

The Problem

- A model asked to prove something will hand back something adjacent and call it done
- It will prove a weaker statement, or reduce the problem to another open one and stop
- It will settle on its first approach and spend the night polishing it
- A human checking a long proof is slower than the model writing one
- The question: how do you write the task so the model cannot cheat, and self-checks?

The Setup

- The prompt restates the problem exactly, and says what a complete proof must show
- It lists the weaker results that do not count, and the traps specific to the problem
- It requires many approaches at once, kept alive even when they contradict each other
- It requires the model to hunt for counterexamples to its own lemmas
- A route that only reduces the problem to another open one is declared blocked
- Separate adversarial agents attack every draft. It runs for a day in a coding agent

Results and Limitations

- Six of about thirteen. The failures cost compute, not a month of anyone's life
- Two proofs were machine-checked by July. The rest are claims until they are
- A similar run, 16 hours by a neurosurgery resident, claims a 2004 conjecture
- Crouzeix checked that proof; a second, independent proof followed eight days later
- The rate is his own count, and someone else paid for 32 hours of compute
- "Open" needs checking. One earlier AI result had been published in 1977
- He picked problems a model could plausibly do. The recipe does not choose

The Method: Specification and Audit

1. Restate the task exactly, and have the model confirm the restatement before starting
2. Say what counts as done. Then say what does not: near-misses, partial results
3. List the traps you already know, so it does not rediscover them
4. Require several routes in parallel, and counterexamples against its own claims
5. Define "blocked": progress that only moves the problem somewhere else does not count
6. Have a second agent attack the result before you see it. Then read the audit

Works for a proof, a literature review, a model spec, a referee report, a pipeline

Named Technique: Chain of Thought

- Asking a model to reason in writing before it answers makes the answer better
- The reasoning modes in today's models do this internally, at length, if you let them
- The effort setting is the knob: here it was turned to the maximum and left for a day
- The written reasoning is also the audit trail: where it went wrong is in the text
- It costs time and tokens, so it belongs on the hard problems, not on every call

Sources: The Cases

- Case 1. Rogic, Mancarci, Xu, Xiao, Yan and Pavlidis, Application of large language models to the annotation of cell lines and mouse strains in genomics data, bioRxiv, 12 March 2026; peer-reviewed in Database (2026). Companion: Rajeev and Narayan, General-purpose large language models can achieve physician-level accuracy in complex medical data extraction, medRxiv, 10 June 2026. Preprint; simulated notes.
- Case 2. Nakamura, Tan and Yean, Observational equivalence of LLM and human annotation, arXiv:2609.22133, August 2026, PolMeth 2026. Companion: Korthals et al., Towards reliable LLM grading through self-consistency and selective human review, Machine Learning and Knowledge Extraction 8(3):74, March 2026.
- Case 3. Spillias et al., Operationalising LLM-assisted screening of literature to support systematic reviews, bioRxiv, July 2026. R package screenllm.
- Case 4. Takahara et al., Hypothesis-driven autonomous materials synthesis with multimodal LLM agents, arXiv:2609.18598, September 2026. Preprint; one material, one variable.
- Case 5. Alexander, Gouskos, Koushiappas, Sun and Niu, CosmoAgent: AI agent for theoretical cosmology, arXiv:2610.16536, 12 October 2026, revised 23 October. Preprint, two weeks old.
- Case 6. Shouqiao Wang, thread on X, 23 July 2026, and `https://github.com/ShouqiaoW/erdos`. Self-reported; two of six proofs in Lean. Jin: preprint 27 July 2026; Townsend and Greenbaum, SIAM News, August 2026; Lorist and Schwenninger, arXiv:2608.03841, 4 August 2026. Neither Crouzeix proof is peer reviewed yet.